

Motor de Bases de Datos SQLite

Alexandra Bernal Escalante
ID: 000474286

Universidad Pontificia Bolivariana
Facultad de Ingeniería de Sistemas e Informática
Desarrollo de Aplicaciones Móviles

Bucaramanga, Colombia
Agosto, 2024

Índice

1. ¿Qué es SQLite?	2
1.1. Definición	2
1.2. Arquitectura	2
1.3. Características y Componentes Principales	4
2. Aplicaciones	4
3. Ventajas y Desventajas	5
3.1. Ventajas de SQLite	5
3.2. Desventajas	6
3.2.1. Limitaciones	6
3.2.2. Problemas Conocidos	7
3.3. Shared Preferences vs SQLite	8

Motor de Bases de Datos SQLite

1. ¿Qué es SQLite?

1.1. Definición

SQLite es una herramienta de software libre, la cual facilita el almacenamiento de información en dispositivos integrados de una manera más sencilla, eficaz, potente, rápida o también en equipos con pocas capacidades de hardware, como un teléfono celular. SQLite implementa el estándar SQL92. [1] Por otra, SQLite está diseñado en base a C, facilitando así la migración para diferentes plataformas de sistemas operativos y dispositivos.

1.2. Arquitectura

A lo contrario de otros gestores de bases de datos, SQLite no utiliza un sistema cliente-servidor, este se integra directamente a la aplicación que lo utiliza. Su arquitectura esta compuesta por cuatro componente principales Core (Núcleo), SQL Compiler (Compilador SQL), Backend y Accessories (Accesorios). [2]

De primer momento la **Interfaz (interface)** es la encargadas de recibir las sentencias SQL provenientes de la aplicación, para enviarlas a **SQL Command Processor (Procesador de Comandos SQL)**, el cual tiene como tarea convertir las sentencias en bytecode con el objetivo que la máquina virtual las pueda entender , para este proceso de primer momento el **Tokenizador (Tokenizer)**, toma las consultas SQL y las dividen en tokens , teniendo las unidades básicas (palabras claves, nombre de la columna, operadores , etc). Ejemplo, se tiene la consulta: **SELECT name, age FROM users WHERE age >= 18**; esta se dividirá de la siguiente manera:

[SELECT] [name] [,] [age] [FROM] [users] [WHERE] [age] [>=] [18] [;]

Figura 1: División de Tokens

Después de lo anterior, la sentencia dividida se manda al **Analizador o Parser**, con el objetivo que mediante *Lemon parser generator* , organice los tokens de forma jerárquica(árbol), como también validar la sintaxis de la consulta, para que finalmente

Code Generator convierta la declaración en bytecode y la envíe a la **máquina virtual**, encargada de realizar las operaciones correspondientes a las consultas.

La estructura de datos implementada por SQLite para almacenar los datos de las tablas e índices son los **B-tree**, los cuales se encuentran todos en un mismo archivo del disco. El **pager o page cache** es el encargado de leer, escribir y almacenar las páginas (unidades de almacenamiento) solicitadas por el módulo del B-tree en el archivo de bases de datos, también garantizar que las solicitudes se manejen de manera rápida, segura y eficiente. Por último, se tiene **OS Interface**, el cual es un objeto abstracto llamado VFS, que tiene como objetivo proporcionar la portabilidad entre sistemas operativos, este incluye diferentes métodos para realizar tareas específicas de acuerdo con el SO que se tenga, SQLite, proporciona VFS para Unix y Windows.

Las **utilidades(utilities)** son herramientas adicionales como la asignación de memoria, las rutinas de comparación de cadenas sin mayúsculas o rutinas portátiles de texto a número, que ayudan a la optimización de la base de datos, ejemplo, la optimización de consultas. Por otra parte, el **código de prueba (Test Code)**, son archivos que no se incluyen en la versión de estándar, pero, se utiliza para verificar que SQLite funcione de manera correcta.

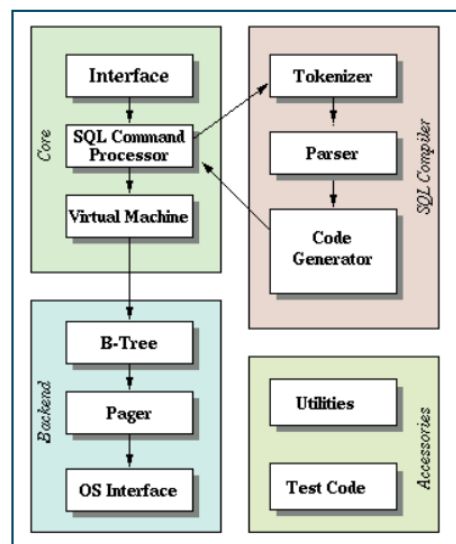


Figura 2: Arquitectura de SQLite. Imagen tomada de [3]

1.3. Características y Componentes Principales

Estas son algunas de las características y componentes principales de SQLite:

- Funciona completamente en la memoria RAM, esto quiero decir que, la base de datos se crea y se gestiona totalmente en ella, en lugar que el disco duro. Este sistema, es ideal para aplicaciones que requieran procesar los datos de manera temporal y la persistencia de datos no es necesaria.
- SQLite tiene un tamaño de archivo binario que suele ser menor a 230k , es por esto que, se utiliza para sistemas embebidos , aplicaciones móviles y entornos en donde su memoria es limitada .
- SQLite es un sistema autónomo o “self-contained”, esto quiere decir que, no tiene dependencias externas, por lo tanto, puede utilizarse en cualquier sistema operativo. Por otra parte, no utiliza bibliotecas o interfaces externas, como también esta encapsulada en un único archivo de código fuente que no requiere instalaciones ni herramientas para su construcción.
- Zero-Configuration Database, esto hace referencia que no es necesario realizar algún proceso de instalación o configuración antes de utilizarlo.
- SQLite es un gestor de bases de datos que no utiliza servidor, más conocido como serveless, esto hace referencia a que no existe un intermediario para la comunicación entre el programa para acceder a la base de datos , la lectura y escritura se realiza directamente a los archivos en el disco.
- Es una base de datos transaccional, esta se define que los datos se almacenan en el disco en filas, en lugar de columnas.[4]

2. Aplicaciones

Gracias a sus características de poder manejar datos estructurados en un solo archivo, SQLite es implementado en diferentes ámbitos en donde es necesario almacenar los datos de forma local o también para sistemas que requieren manejar datos sin la necesidad de gestionar un sistema de bases de datos completo [5] La siguiente lista, representa los contextos en donde mejor implementar SQLite.

- Se utiliza en aplicaciones que requieren una base de datos integrada en el mismo sistema, ejemplo, aplicaciones de mensajería, almacenamiento de configuraciones de usuarios, y gestiones de datos sin conexión.
- También se implementa para entornos de prueba para sistemas de administración de bases de datos relacionadas (RDBMS), esto con el objetivo de ahorrar recursos.
- En el ámbito empresarial, se utiliza como cache local en el manejador de bases de datos, con el objetivo de acelerar el tiempo de respuesta y reducir la carga que se tenga a la base de datos central.
- SQLite funciona como motor de bases de datos para sitios web en los cuales exista un tráfico bajo, principalmente aquellos que reciban un estimado de 100,000 visitas al día, cabe aclarar que no es un límite máximo, ya que se ha comprobado que puede manejar alrededor de 400.00 a 500,000 solicitudes HTTP por día.
- Debido a que SQLite no requiere ningún tipo de administración, es ideal para sistemas embebidos, como dispositivos IoT, ya que estos requieren operar sin la necesidad de soporte humano. Por otra parte, como SQLite tiene un bajo consumo de recursos, es ideal para estos casos, ya que ayuda a operar de manera eficiente con poca energía y capacidad de procesamiento.
- SQLite puede utilizarse como contenedor de datos, con el objetivo de sustituir los archivos ZIP o Tar, ya que este puede implementar actualizaciones incrementales y atómicas, como también la capacidad de almacenar metadatos.

3. Ventajas y Desventajas

3.1. Ventajas de SQLite

- **Software Libre:** Debido a que es software libre, se tiene la posibilidad de encontrar gran cantidad de componentes, librerías y drivers en distintos lenguajes de programación que sean compatibles con SQLite, facilitando la implementación en distintos ámbitos [6], ya sea en lenguajes modernos como Java, Perl, Python, PHP, Ruby, C#, o lenguajes más antiguos como Pascal, SmallTalk, Clipper.

- **Mayor rendimiento:** La escritura y lectura desde la base de datos SQLite es más rápida que hacerlo por medio de archivos individuales de un disco, trayendo consigo mejor rendimiento y respuestas rápidas para el aplicativo, debido a que la aplicación solo carga la información o datos que necesite , en vez de tener que leer un archivo completo de memoria.
- **Altamente Confiable:** El propósito principal de las bases de datos es mantener la información organizada y segura, es por esto que, antes de lanzar algún tipo de actualización o versión de SQLite, su core pasa por una serie de pruebas con el objetivo de poder detectar sus vulnerabilidades o fallas que pueda presentar, también para medir el estado de la memoria y el límite de almacenamiento que puede soportar. De pruebas unitarias y pruebas de peticiones, se realizan alrededor de 10 millones. [1]

3.2. Desventajas

3.2.1. Limitaciones

A pesar de las grandes ventajas que proporciona SQLite, existen entornos en donde esta tecnología no se puede implementar, debido su estructura y arquitectura, estos casos son los siguientes. [7]

- **Aplicaciones Cliente/Servidor:** SQLite no es recomendable para ámbitos en donde sea necesario acceder simultáneamente a la base de datos por medio de múltiples dispositivos a través de una red, ya que este no presenta un bloque de archivos eficientes, lo que puede provocar que dos o más clientes intenten modificar la misma parte de la base de datos al mismo tiempo, trayendo consigo corrupción de datos.
- **Sitios Web de gran Volumen:** Aunque SQLite funciona correctamente como la base de datos backend para sitios web, para aquellos que presenta una alta demanda de peticiones de escritura y se requieren múltiples servidores para responder, es mejor optar una base de datos que utilice cliente/servidor., con el fin que su respuesta sea rápida y optimizada.
- **Conjuntos de Datos grandes:** SQLite está limitada en tamaño a 281 terabytes

(248 bytes, 256 tebibytes), por otra parte, almacena toda la base de datos en un solo archivo de disco y muchos sistemas de archivos limitan el tamaño máximo a algo menor que esto, es por esto que , no es la opción más viable para almacenar bases de datos que requieran una gran magnitud , porque puede afectar su rendimiento y respuesta.

- **Alta Concurrencia:** SQLite soporta un número ilimitado de lectores simultáneos, pero solo permite un escritor a la vez, y estos se ponen en cola para poder realizar la acción. Es por esto, que para aplicaciones que tienen un alto grado de concurrencia, es necesario buscar opciones que permitan lo anterior.

3.2.2. Problemas Conocidos

SQLite a pesar de ser conocida como una de las bases de datos más usadas en los diferentes ámbitos de la tecnología y su primera versión fue lanzada en el año 2000, no está extensa a presentar fallas y vulnerabilidades que pueda afectar su rendimiento.

En los últimos 10 años se han descubierto una serie de problemas que puede afectar la integridad de los datos. Uno de ellos fue en el 2016 descubierta por KoreLogic [8], en donde la versión 3.13.0 presentaba un fallo en la seguridad en donde se comprometía los datos personales de los usuarios, debido a no configurar los archivos temporales en una ruta segura.

Por otra parte, de los fallos más conocidos de SQLite fue en el año 2018, conocido como *Magellan*, en donde le permitía al atacante ejecutar un código arbitrario, así como también acceder a la memoria de la app y que esta se cierre de manera inesperada.[9]

En octubre de 2022, Trail of Bits publicó un análisis sobre una vulnerabilidad en SQLite, identificada como *CVE-2022-35737*. Esta hace referencia al fallo de la función `sqlite3_snprintf`, utilizada en programas C/C++ que trabajan con SQLite. Si se pasa una cadena de caracteres extremadamente grande (más de 2 GB), puede causar que el programa se bloquee, lo que podría permitir un ataque de denegación de servicio (DoS) o incluso la ejecución de código arbitrario[10]. Aunque el fallo se introdujo hace 22 años, cuando los parámetros grandes no eran comunes, no fue detectado antes debido a las limitaciones en las técnicas de prueba.

3.3. Shared Preferences vs SQLite

SQLite y SharedPreferences son dos métodos diferentes para almacenar datos en aplicaciones Android, cada uno con características y usos distintos. SQLite es una base de datos relacional embebida que maneja datos estructurados en tablas con filas y columnas, permitiendo consultas SQL complejas y transacciones ACID. Es ideal para aplicaciones que requieren gestionar grandes volúmenes de datos o datos complejos, y es adecuado para aplicaciones móviles, software de escritorio y dispositivos embebidos.

SharedPreferences, por otro lado, es un método de almacenamiento en pares clave-valor, diseñado para guardar configuraciones y preferencias del usuario.[11] Es muy simple de usar y configurar, adecuado para datos pequeños y operaciones rápidas, pero no soporta transacciones ni consultas avanzadas.

La base de datos de SQLite se guarda en un solo archivo, mientras que SharedPreferences almacena datos en un archivo XML en Android. SQLite también maneja múltiples lectores simultáneamente y un solo escritor, lo que es útil para aplicaciones con alta concurrencia, mientras que SharedPreferences no está diseñado para manejar concurrencia o sincronización entre múltiples dispositivos.

Referencias

- [1] J. Kreibich, *Using SQLite*. O'Reilly Media, Inc., 2010.
- [2] Runebook, “Arquitectura de sqlite,” 2024. Accessed: 2024-08-23.
- [3] SQLite, “Architecture of sqlite,” 2024. Accessed: 2024-08-23.
- [4] G. Cloud, “What are transactional databases?,” 2024. Accessed: 2024-08-23.
- [5] SQLite, “Situations where SQLite works well.” https://www.sqlite.org/whentouse.html#situations_where_sqlite_works_well, 2024. Accessed: 2024-08-23.
- [6] SG Software Guru, “Sqlite: La base de datos embebida.” <https://sg.com.mx/revista/17/sqlite-la-base-datos-embebida>, 2024. Accessed: 2024-08-23.
- [7] S. Bhosale, T. Patil, and P. Patil, “Sqlite: Light database system,” *Int. J. Comput. Sci. Mob. Comput*, vol. 44, no. 4, pp. 882–885, 2015.
- [8] RedesZone, “Una vulnerabilidad en SQLite expone datos personales de los usuarios.” <https://www.redeszone.net/2016/07/08/una-vulnerabilidad-sqlite-expone-datos-personales-los-usuarios/>, jul 2016. Accessed: 2024-08-23.
- [9] M. López, “Un fallo en la base de datos SQLite afecta a millones de apps y a los sistemas de apple.” <https://www.applesfera.com/profesional/fallo-base-datos-sqlite-afecta-a-millones-apps-a-sistemas-apple>, 2024. Accessed: 2024-08-23.
- [10] National Institute of Standards and Technology (NIST), “Cve-2022-35737 detail.” <https://nvd.nist.gov/vuln/detail/CVE-2022-35737>, 2024. Accessed: 2024-08-23.
- [11] GeeksforGeeks, “Shared preferences in android with examples,” 2024. Accessed: 2024-08-23.